Date: 06-04-2026

Objects:

objects are used to store keyed collections of various data and more complex entities. In JavaScript, objects penetrate almost every aspect of the language.

An object can be created with curly braces {…} with an optional list of *properties*. A property is a “key: value” pair, where key is a string (also called a “property name”), and value can be anything.

An empty object (“empty cabinet”) can be created using one of two syntaxes:

```
let user = new Object(); // "object constructor" syntax

let user = {};  // "object literal" syntax
```

Literals and properties:

```
let user = {     // an object

  name: "John",  // by key "name" store value "John"

  age: 30        // by key "age" store value 30

};
```

Here we can access the values using these syntax:

```
User.age ( object_name.key )
```

And also

```
"likes birds": true  // multiword property name must be quoted
```

Square brackets:

If I want to access the multi word property with dot operator its not possible

```
user.likes birds = true // the result will be undefined

let user = {};

// set

user["likes birds"] = true;

// get

alert(user["likes birds"]); // true

// delete

delete user["likes birds"];
```

If we try to the access like providing a dot operator before assigning a key to a variable its shows undefined

Like in the example below

```
let user = {
  name: "John",
  age: 30
};


let key = "name";
alert( user.key ) // undefined
```

Computed Properties:

We can use square brackets in an object literal, when creating an object. That's called *computed properties*.

```
let fruit = prompt("Which fruit to buy?", "apple");
let bag = {
  [fruit]: 5, // the name of the property is taken from the variable fruit
};
alert( bag.apple ); // 5 if fruit="apple"
```

So most of the time, when property names are known and simple, the dot is used. And if we need something more complex, then we switch to square brackets.

There is no limitations for the property names.

Property existence test, "in" operator:

The syntax is:

```
"key" in object
let user = { name: "John", age: 30 };
alert( "age" in user ); // true, user.age exists
alert( "blabla" in user ); // false, user.blabla doesn't exist
```

The for-in loop:

```
for (key in object) {
// executes the body for each key among object properties
}
```

```
let user = {
  name: "John",
  age: 30,
  isAdmin: true
```

```
};
for (let key in user) {
  // keys
  alert( key );  // name, age, isAdmin
  // values for the keys
  alert( user[key] ); // John, 30, true
}
```

- The dot notation: obj.property.
- Square brackets notation obj["property"]. Square brackets allow taking the key from a variable, like obj[varWithKey].

Object references and copying:

Objects in JavaScript are copied by reference, NOT by value

Cloning objects :

For this we can clone manually or using object.assign().

```
let clone = Object.assign({}, user);
```

Shallow copy can be done using Object.assign() or spread operator.

```
let clone = Object.assign({}, obj);
// or
let clone = { ...obj };
```

Shallow copy shares **nested objects**.

```
obj.nested.x = 5; // affects clone
```

Deep copy can be done using structuredClone().

```
let clone = structuredClone(obj);
```

The functions can not be cloned.

Garbage collection:

When we create a variable,object or function created the memory will be allocated for that after the program ends we need clear that memory that memory is called garbage collection.

The main concept of memory management in JavaScript is *reachability*.

Here the reachable values refers to the,

For example, if a object is references the variable then that variable is reachable if the reence cuts or no longer exist then we need clear that unreachable variable.

For that in javscript they are using a basic algorithm called mark-and -sweep.

Mark and sweep alogorithm:

Step 1 : Marks the roots

Step 2: Then we follow their references and mark referenced objects

Step 3: And continue to follow further references, while possible

Step 4: Now the objects that could not be visited in the process are considered unreachable and will be removed

That's the concept of how garbage collection works. JavaScript engines apply many optimizations to make it run faster and not introduce any delays into the code execution.

Some of the optimizations:

- **Generational collection** – objects are split into two sets: "new ones" and "old ones". In typical code, many objects have a short life span: they appear, do their job and die fast, so it makes sense to track new objects and clear the memory from them if that's the case. Those that survive for long enough, become "old" and are examined less often.
- **Incremental collection** – if there are many objects, and we try to walk and mark the whole object set at once, it may take some time and introduce visible delays in the execution. So the engine splits the whole set of existing objects into multiple parts. And then clear these parts one after another. There are many small garbage collections instead of a total one. That requires some extra bookkeeping between them to track changes, but we get many tiny delays instead of a big one.
- **Idle-time collection** – the garbage collector tries to run only while the CPU is idle, to reduce the possible effect on the execution.
- Functions that are stored in object properties are called "methods".
- Methods allow objects to "act" like object.doSomething().
- Methods can reference the object as this.

The value of this is defined at run-time.

- When a function is declared, it may use this, but that this has no value until the function is called.
- A function can be copied between objects.
- When a function is called in the "method" syntax: object.method(), the value of this during the call is object.

note that: arrow functions are special: they have no this. When this is accessed inside an arrow function, it is taken from outside.

Constructor function:

- Constructor functions or, briefly, constructors, are regular functions, but there's a common agreement to name them with capital letter first.

- Constructor functions should only be called using new. Such a call implies a creation of empty this at the start and returning the populated one at the end.

We can use constructor functions to make multiple similar objects.

JavaScript provides constructor functions for many built-in language objects: like Date for dates, Set for sets and others that we plan to study.

```
function User(name) {
  this.name = name;
  this.isAdmin = false;
}
let user = new User("Jack");
alert(user.name); // Jack
alert(user.isAdmin); // false
```

Optional chaining using ?:

This is one of the method to get null if the value is not there

```
// document.querySelector('.elem') is null if there's no element
let html = document.querySelector('.elem').innerHTML; // error if it's null
```

This will return undefined if the user didn't gave the value

```
let user = {};
alert(user.address ? user.address.street : undefined);
```

using chained ?

```
let user = {}; // user has no address
alert(user.address ? user.address.street ? user.address.street.name : null : null);
```

We can also do this using &&

```
let user = {}; // user has no address
alert( user.address && user.address.street && user.address.street.name ); // undefined (no error)
```

AND'ing the whole path to the property ensures that all components exist (if not, the evaluation stops), but also isn't ideal.

As you can see, property names are still duplicated in the code. E.g. in the code above, user.address appears three times.

That’s why the optional chaining ?. was added to the language. To solve this problem once and for all!

Optional Chaining:

The optional chaining ?. stops the evaluation if the value before ?. is undefined or null and returns undefined.

The safe way to access user.address.street using ?.:

```
let user = {}; // user has no address

alert( user?.address?.street ); // undefined (no error)
```

The optional chaining ?. syntax has three forms:

1. obj?.prop – returns obj.prop if obj exists, otherwise undefined.
2. obj?.[prop] – returns obj[prop] if obj exists, otherwise undefined.
3. obj.method?.() – calls obj.method() if obj.method exists, otherwise returns undefined.